

Detailed Balance Normalizing Flow for Generative Sampling

Qi Feng¹, Rongjie Lai², and Xuda Ye²

¹Department of Mathematics, Florida State University, Tallahassee, FL 32306

²Department of Mathematics, Purdue University, West Lafayette, IN 47907

May 3, 2026

1 Introduction

Normalizing flows based on the KL divergence (Rezende and Mohamed, 2015) have been widely applied in generative sampling. In energy-based sampling from physical potentials, the Boltzmann generator (Noé et al., 2019; Wu et al., 2020) has been the industry standard for fast sampling from target distributions. For data-driven sampling problems, although normalizing flows are less popular than score-based diffusion (Ho et al., 2020; Song et al., 2021) and flow matching (Lipman et al., 2023), they remain a versatile tool across a range of applications, including density estimation (Papamakarios et al., 2017; Kingma and Dhariwal, 2018), simulation-based inference (Papamakarios et al., 2019), and anomaly detection (Kirichenko et al., 2020; Rudolph et al., 2021).

Despite their popularity, KL normalizing flows suffer from several intrinsic drawbacks. First, the flow map is a diffeomorphism on the whole space, so for multi-modal distributions the resulting NF often introduces artificial bridges between modes. Second, the KL loss is highly sensitive to mode collapse, and the flow easily falls into a few dominant modes while neglecting the rest.

In this paper, we propose a normalizing flow based on the detailed balance (DB) divergence, which serves as an alternative for multi-modal generative sampling tasks. Its key advantage is that the DB divergence matches the gradient rather than the function value and keeps the loss function nonnegative. This incurs only a constant-factor overhead in training while offering substantially more flexibility in use.

2 Detailed balance divergence

2.1 Basic definition

We introduce the detailed balance (DB) loss to characterize the discrepancy between two distributions. Let μ and ν be probability distributions on $\mathbb{R}^d$. We define the detailed balance (DB) divergence as

$$\text{DB}(\mu\|\nu) = \int_{\mathbb{R}^d} \mu(x) \left| \nabla \left(\frac{\nu(x)}{\mu(x)} \right) \right| dx, \quad (1)$$

which is directly reminiscent of the KL divergence. The DB divergence is nonsymmetric and nonnegative by definition, and it enforces $\mu(x) = \nu(x)$ by driving the ratio $\nu(x)/\mu(x)$ toward a constant. This is fundamentally different from the KL divergence, whose nonnegativity relies on Jensen’s inequality.

The DB divergence (1) can be equivalently written as

$$\text{DB}(\mu\|\nu) = \int_{\mathbb{R}^d} \nu(x) |\nabla \log \mu(x) - \nabla \log \nu(x)| dx. \quad (2)$$

If $\mu(x)$ takes the unnormalized potential form $\mu(x) \propto e^{-U(x)}$, where $U(x)$ is the potential function, then

$$\text{DB}(\mu\|\nu) = \int_{\mathbb{R}^d} |\nabla \nu(x) + \nu(x) \nabla U(x)| dx.$$

Therefore, minimizing the DB divergence amounts to solving the PDE

$$\nabla \nu(x) + \nu(x) \nabla U(x) = 0, \quad (3)$$

which is exactly the detailed balance condition for sampling $e^{-U(x)}$ via Langevin dynamics. The DB divergence therefore carries a natural physical meaning.

2.2 Relations to other distribution metrics

The DB divergence is closely related to two classical distribution metrics, the Fisher divergence and the total variation distance, and sits naturally between the two in strength.

Fisher divergence. The Fisher divergence between μ and ν is

$$\text{Fisher}(\nu\|\mu) = \int_{\mathbb{R}^d} \nu(x) |\nabla \log \mu(x) - \nabla \log \nu(x)|^2 dx,$$

so by the equivalent form (2), the DB divergence is precisely its L^1 counterpart, with the squared L^2 norm of the score difference replaced by the L^1 norm. Cauchy–Schwarz together with $\int \nu(x) dx = 1$ then gives

$$\text{DB}(\mu\|\nu)^2 \leq \text{Fisher}(\nu\|\mu). \quad (4)$$

Total variation distance. The total variation distance between μ and ν is

$$\text{TV}(\mu, \nu) = \frac{1}{2} \int_{\mathbb{R}^d} |\mu(x) - \nu(x)| dx.$$

Set $f(x) = \nu(x)/\mu(x) - 1$, so that $\int \mu(x) f(x) dx = 0$ since μ and ν are normalized. Then

$$\text{TV}(\mu, \nu) = \frac{1}{2} \int_{\mathbb{R}^d} \mu(x) |f(x)| dx, \quad \text{DB}(\mu\|\nu) = \int_{\mathbb{R}^d} \mu(x) |\nabla f(x)| dx.$$

If μ satisfies a Cheeger inequality (an L^1 Poincaré inequality) with isoperimetric constant $c_P > 0$, applying it to the mean-zero function f yields

$$\text{TV}(\mu, \nu) \leq \frac{1}{2c_P} \text{DB}(\mu\|\nu). \quad (5)$$

Such a Cheeger inequality holds for log-concave distributions and Gaussian mixtures with sufficient overlap, so the DB divergence controls the total variation distance for these classes.

Combining (4) and (5) gives the hierarchy

$$\text{TV}(\mu, \nu) \leq \frac{1}{2c_P} \text{DB}(\mu\|\nu) \leq \frac{1}{2c_P} \sqrt{\text{Fisher}(\nu\|\mu)}, \quad (6)$$

placing the DB divergence between total variation and Fisher divergence. It is weaker than the Fisher divergence yet still strong enough to control total variation under mild regularity assumptions on μ .

To illustrate the hierarchy (6), consider the one-dimensional family

$$\mu_\sigma \propto e^{-U_\sigma(x)}, \quad U_\sigma(x) = \frac{1}{2}x^2 + \frac{1}{\sqrt{2\pi}} e^{-\frac{x^2}{2\sigma^2}}, \quad \nu \propto e^{-V(x)}, \quad V(x) = \frac{1}{2}x^2,$$

indexed by $\sigma \in \{2^0, 2^{-1}, \dots, 2^{-7}\}$. The potentials U_σ and V differ by a Gaussian-shaped bump whose total mass equals σ , so as $\sigma \rightarrow 0$ the perturbation becomes thinner and taller but carries less and less probability mass. Figure 1 shows that the three metrics react to this in very different ways: KL decays at the rate of the bump’s mass and effectively ignores the perturbation; DB treats it as a fixed-size disturbance and stays roughly constant; Fisher amplifies the steep gradient and diverges.

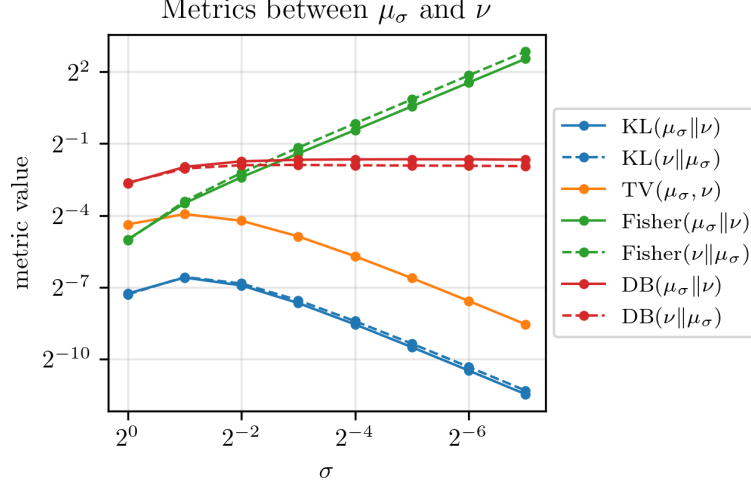


Figure 1: The four metrics between μ_σ and ν for $\sigma = 2^0, 2^{-1}, \dots, 2^{-7}$. KL and TV decay with σ ; DB stays roughly constant; Fisher diverges.

3 Energy-based DB normalizing flow

Let $U_0(x)$ and $U_1(x)$ be source and target potential functions on $\mathbb{R}^d$, and let $\mu_0(x) \propto \exp(-U_0(x))$ and $\mu_1(x) \propto \exp(-U_1(x))$ be the corresponding Boltzmann distributions. Given explicit samples from μ_0 , we aim to train a flow map F such that $F_\# \mu_0 = \mu_1$, where the pushforward distribution has the density

$$(F_\# \mu_0)(y) = \frac{\mu_0(x)}{|\det J_F(x)|}, \quad y = F(x). \quad (7)$$

where $J_F(x) \in \mathbb{R}^d$ is the Jacobian matrix of the flow F . The the loss function built from the DB divergence between the distributions $F_\# \mu_0$ and μ_1 .

We first show how to construct the DB loss function and derive the training for DB normalizing flows. We then introduce the DB-based Boltzmann generator by combining a sequence of DB normalizing flows.

3.1 Construction of loss function

First, we consider the forward DB divergence $\text{DB}(\mu_1 \| F_\# \mu_0)$. Define the log importance weights by

$$W(x) = \log \frac{(F_\# \mu_0)(y)}{\mu_1(y)} + \text{const} = U_1(F(x)) - U_0(x) - \log |\det J_F(x)|, \quad (8)$$

then the DB divergence can be explicitly computed as

$$\begin{aligned} \text{DB}(\mu_1 \| F_\# \mu_0) &= \int_{\mathbb{R}^d} \mu_1(y) \left| \nabla_y \left(\frac{(F_\# \mu_0)(y)}{\mu_1(y)} \right) \right| dy = \text{const} \cdot \int_{\mathbb{R}^d} e^{-U_1(y)} |\nabla_y (e^{W(x)})| dy \\ &= \text{const} \cdot \int_{\mathbb{R}^d} e^{-U_1(y) + W(x)} |\nabla_y W(x)| dy. \end{aligned}$$

Applying the change-of-variable formula with $y = F(x)$, we obtain

$$\text{DB}(\mu_1 \| F_\# \mu_0) = \int_{\mathbb{R}^d} \frac{\mu(x)}{|\det J_F(x)|} |\nabla_y \mathcal{R}_F(x)| dy = \int_{\mathbb{R}^d} \mu(x) |\nabla_y \mathcal{R}_F(x)| dx. \quad (9)$$

Therefore, computing the DB loss (9) requires evaluating the gradient of the log-likelihood ratio $\mathcal{R}_F(x)$ defined in (8) with respect to the implicit variable y .

Given the expression of $\mathcal{R}_F(x)$ in (8), a direct application of the chain rule yields

$$\begin{aligned}\nabla_y \mathcal{R}_F(x) &= \nabla U_0(F(x)) - \nabla_y (U(x) + \log |\det J_F(x)|) \\ &= \nabla U_0(F(x)) - J_F(x)^{-\top} \left(\nabla U(x) + \nabla (\log |\det J_F(x)|) \right).\end{aligned}\quad (10)$$

Substituting (10) into (9), the loss function in terms of the flow map F takes the form

$$\text{DB}(\mu_0 \| F_{\#} \mu) = \int_{\mathbb{R}^d} \mu(x) \left| \nabla U_0(F(x)) - J_F(x)^{-\top} \left(\nabla U(x) + \nabla (\log |\det J_F(x)|) \right) \right| dx. \quad (11)$$

The loss function (11) is a highly nonlinear functional of F , primarily due to the explicit presence of the Jacobian matrix $J_F(x)$ (rather than merely its determinant). Standard normalizing flow architectures such as RealNVP and NSF only support efficient computation of the determinant; computing the full Jacobian requires at least $O(d^2)$ cost. In high dimensions, this becomes a significant computational burden, even with automatic differentiation over the parameters of F .

To address this, we propose a finite-difference approach to approximate the loss function while keeping the computational cost of evaluating $\nabla_y \mathcal{R}_F(x)$ at $O(d)$. We choose a small step size $\varepsilon > 0$ and a random direction $v \in \mathbb{S}^{d-1}$, and consider the two perturbed points $F(x) \pm \varepsilon v$ near $F(x)$. By central differencing, we obtain the approximations

$$v^\top \nabla U_0(F(x)) \approx \frac{U_0(F(x) + \varepsilon v) - U_0(F(x) - \varepsilon v)}{2\varepsilon} \quad (12)$$

and

$$J_F(x)^{-1} v \approx \frac{F^{-1}(F(x) + \varepsilon v) - F^{-1}(F(x) - \varepsilon v)}{2\varepsilon}. \quad (13)$$

Substituting (12) and (13) into the expression for $\nabla_y \mathcal{R}_F(x)$ in (10), we obtain the directional derivative approximation

$$\begin{aligned}v^\top \nabla_y \mathcal{R}_F(x) &\approx \frac{U_0(F(x) + \varepsilon v) - U_0(F(x) - \varepsilon v)}{2\varepsilon} \\ &\quad - \left(\frac{F^{-1}(F(x) + \varepsilon v) - F^{-1}(F(x) - \varepsilon v)}{2\varepsilon} \right)^\top \left(\nabla U(x) + \nabla (\log |\det J_F(x)|) \right).\end{aligned}\quad (14)$$

With this approximation, the loss (9) can be estimated by repeatedly sampling x from the target distribution $\mu(x) \propto e^{-U(x)}$ and v uniformly from the unit sphere $\mathbb{S}^{d-1}$.

In particular, the loss (9) does not depend on the normalizing constant of μ_0 , and thus allows a flexible choice of the prior distribution.

3.2 Alternate construction with the prior as the source

When the prior $\mu_0(x) \propto e^{-U_0(x)}$ is chosen close to the target $\mu(x) \propto e^{-U(x)}$, it is natural to train the flow map $F: \mathbb{R}^d \rightarrow \mathbb{R}^d$ in the conventional direction, so that $F_{\#} \mu_0 \approx \mu$, as in the standard energy-based normalizing flow. The pushforward density reads

$$(F_{\#} \mu_0)(y) = \frac{\mu_0(x)}{|\det J_F(x)|}, \quad y = F(x).$$

We adopt $\text{DB}(\mu \| F_{\#} \mu_0)$ as the loss. The corresponding log-likelihood ratio is

$$\mathcal{R}_F(x) = \log \frac{(F_{\#} \mu_0)(y)}{\mu(y)} + \text{const} = U(F(x)) - U_0(x) - \log |\det J_F(x)|, \quad (15)$$

which differs from (8) only by interchanging the roles of U and U_0 . Repeating the change-of-variable argument that led to (9) yields

$$\text{DB}(\mu \| F_{\#} \mu_0) = \int_{\mathbb{R}^d} \mu(y) \left| \nabla \left(\frac{(F_{\#} \mu_0)(y)}{\mu(y)} \right) \right| dy = \int_{\mathbb{R}^d} \mu_0(x) |\nabla_y \mathcal{R}_F(x)| dx. \quad (16)$$

The right-hand integral is taken against the prior μ_0 , so the loss can be estimated by sampling directly from μ_0 , without requiring samples from the target μ .

The directional derivative of $\mathcal{R}_F$ admits the same finite-difference approximation as in (14), with U and U_0 swapped:

$$v^\top \nabla_y \mathcal{R}_F(x) \approx \frac{U(F(x) + \varepsilon v) - U(F(x) - \varepsilon v)}{2\varepsilon} - \left(\frac{F^{-1}(F(x) + \varepsilon v) - F^{-1}(F(x) - \varepsilon v)}{2\varepsilon} \right)^\top \left(\nabla U_0(x) + \nabla (\log |\det J_F(x)|) \right). \quad (17)$$

This yields a loss that uses no samples from the target. It is precisely the energy-based normalizing flow, with the DB divergence replacing the usual reverse KL: the loss is built from the score difference rather than the log-density ratio, and remains non-negative regardless of sampling bias.

3.3 Adaptive choice of prior distribution

A key practical challenge in evaluating the loss (9) is that it requires sampling from the target distribution $\mu(x) \propto e^{-U(x)}$, which is generally unavailable. However, as noted earlier, the DB divergence is non-negative by construction and remains valid even when μ is sampled with bias. This motivates the following importance sampling strategy:

1. Initialize N_0 particles $x_0^1, \dots, x_0^{N_0}$ from a broad base distribution and evolve them independently via Langevin dynamics

$$dX_t = -\nabla U(X_t)dt + \sqrt{2}dB_t$$

where B_t is the standard Brownian motion in $\mathbb{R}^d$. After sufficient burn-in, the particles $x_0^1, \dots, x_0^{N_0}$ are expected to capture the local modes of μ .

2. Choose an appropriate width parameter $\bar{\sigma}_0 > 0$ and define the Gaussian mixture

$$\bar{\mu}_0(x) = \frac{1}{N_0} \sum_{i=1}^{N_0} \mathcal{N}(x; x_0^i, \bar{\sigma}_0^2),$$

so that $\bar{\mu}_0(x)$ covers nearly all the probability mass of $\mu(x)$.

3. Rewrite the loss (9) via importance weighting as

$$\text{DB}(\mu_0 \| F_{\#}\mu) = \int_{\mathbb{R}^d} \bar{\mu}_0(x) \cdot \frac{\mu(x)}{\bar{\mu}_0(x)} \cdot |\nabla_y \mathcal{R}_F(x)| dx = \mathbb{E}_{x \sim \bar{\mu}_0} \left[\frac{\mu(x)}{\bar{\mu}_0(x)} |\nabla_y \mathcal{R}_F(x)| \right].$$

In this way, the DB loss can be computed by sampling from a fixed, tractable Gaussian mixture $\bar{\mu}_0(x)$ rather than the intractable target $\mu(x)$. Moreover, even if the importance weights $\mu(x)/\bar{\mu}_0(x)$ are computed inaccurately, the DB loss remains non-negative by construction, so the flow map F is still trained towards the correct minimizer.

We emphasize that the fixed importance sampling distribution $\bar{\mu}_0(x)$ is distinct from the flow prior $\mu_0(x)$. The former is used solely for tractable evaluation of the loss, while the latter can be adapted during training to better match the target $\mu(x)$. Specifically, we parameterize $\mu_0(x)$ as

$$\mu_0(x) \propto \sum_{i=1}^{N_0} w_i \mathcal{N}(x; x_0^i, \sigma_i^2), \quad (18)$$

or equivalently, the prior potential function is given by

$$U_0(x) = -\log \mu_0(x) = -\log \left(\sum_{i=1}^{N_0} w_i \mathcal{N}(x; x_0^i, \sigma_i^2) \right). \quad (19)$$

Here, the positions $\{x_0^i\}_{i=1}^{N_0}$ are kept fixed, while the weights $\{w_i\}_{i=1}^{N_0}$ and widths $\{\sigma_i\}_{i=1}^{N_0}$ are treated as trainable parameters. Crucially, the weights w_i do not need to be normalized, since the DB divergence is invariant under rescaling of the prior (cf. Section 2.1). This allows us to optimize the weight parameters freely. The final loss function is then

$$\text{Loss}(F, \{w_i\}_{i=1}^{N_0}, \{\sigma_i\}_{i=1}^{N_0}) = \mathbb{E}_{x \sim \mu_0} \left[\frac{\mu(x)}{\mu_0(x)} |\nabla_y \mathcal{R}_F(x)| \right] + \text{PriorPotential}(\{w_i\}_{i=1}^{N_0}, \{\sigma_i\}_{i=1}^{N_0}), \quad (20)$$

where the prior potential on w_i and σ_i can be conveniently specified via a Gamma distribution. Note that if F were perfectly trained to minimize the DB loss, the total loss (20) would be minimized exactly when all w_i and σ_i coincide at their prior mean, ensuring the theoretical stability of the optimization.

Remark *Since the Gaussian distribution has unbounded support, in practice one may replace it with an isotropic cosine cutoff distribution of finite width, which is more compatible with NSF architectures that operate on a bounded domain.*

3.4 One-step importance sampling at inference

After training with the total loss (20), we obtain the trained flow map F along with the optimized parameters $\{w_i\}_{i=1}^{N_0}$ and $\{\sigma_i\}_{i=1}^{N_0}$, which define a tractable Gaussian mixture prior μ_0 . Since the log-likelihood ratio $\mathcal{R}_F(x)$ is available in closed form via (8), we can sample from the target distribution $\mu(x)$ using the following procedure:

1. Draw N samples $y_1, \dots, y_N$ from the prior distribution μ_0 .
2. Compute the inverse flow positions $x_i = F^{-1}(y_i)$ for $i = 1, \dots, N$.
3. Evaluate the log-likelihood ratio $\mathcal{R}_F(x_i) = U_0(y_i) - U(x_i) - \log |\det J_F(x_i)|$ for $i = 1, \dots, N$.
4. Compute the importance weights $w_i = \exp(\mathcal{R}_F(x_i))$ for $i = 1, \dots, N$ and normalize so that $\sum_{i=1}^N w_i = 1$.
5. The resulting weighted empirical distribution

$$\hat{\mu}(x) = \sum_{i=1}^N w_i \delta(x - x_i)$$

provides an approximation to the target distribution $\mu(x)$. If samples with uniform weights are desired, one may resample from $\hat{\mu}$.

This procedure requires only a single flow evaluation per sample. Its effectiveness relies on the assumption that the adapted prior μ_0 already captures the dominant structure of the target distribution μ , so that the importance weights remain well-behaved.

4 Implementation

The implementation uses `PyTorch` as the primary framework and the `zuko` package for the neural spline flow (NSF) architecture. The Detailed Balance Normalizing Flow (DBNF) method is organized as a standalone Python package `dbnf` with the following source files:

- `potential.py`. Defines a base `Potential` class and specific potential subclasses (e.g., double-well, Gaussian mixture). Each subclass stores the potential name and parameters, and supports batched evaluation of $U(x)$. The gradient $\nabla U(x)$ is computed via `torch` automatic differentiation.
- `distribution.py`. Implements tractable distribution classes such as Gaussian mixtures and cosine cutoff distributions. Log-densities are evaluated using the log-sum-exp trick for numerical stability. Also provides the prior potential regularizer for the adaptive parameters $\{w_i\}$ and $\{\sigma_i\}$, as well as utilities for resampling and computing the effective sample size (ESS).

- `flow.py`. Wraps `zuko`’s NSF implementation, exposing the forward map F , its inverse F^{-1} , and the log-determinant $\log |\det J_F(x)|$.
- `dbloss.py`. Evaluates the DB loss $\text{DB}(\mu_0 \| F_{\#}\mu)$ given the prior μ_0 , the target potential U , and the flow map F , using the finite-difference approximation (14).

Outside the package, the full pipeline consists of the following scripts:

1. `global_parameters.py`. Specifies all global configuration, including: choice of target potential and dimension d ; number of particles N_0 and burn-in steps for the Langevin mode-finding stage; initial width $\bar{\sigma}_0$ for the importance sampling distribution $\bar{\mu}_0$; finite-difference step size ε and number of random directions per sample for evaluating (14); NSF architecture settings (number of layers, hidden dimensions, spline bins); learning rates and number of epochs for both the flow optimizer and the prior optimizer; and output paths.
2. `set_prior.py`. Constructs the importance sampling distribution $\bar{\mu}_0(x)$ by running short Langevin trajectories, and initializes the adaptive prior parameters $\{w_i\}$ and $\{\sigma_i\}$.
3. `train_flow.py`. Combines the DB loss and the prior potential regularizer into the total loss (20). Training proceeds by alternating two optimizers: one updates the flow parameters of F , and the other updates the adaptive prior parameters $\{w_i\}$ and $\{\sigma_i\}$. This alternating scheme decouples the flow and prior updates, allowing separate learning rates and improving training stability.
4. `generate_samples.py`. Performs one-step importance sampling (Section 3.3), generates target samples, and reports the ESS.
5. `plot_results.py`. Visualizes the generated samples and diagnostic statistics.

5 Detailed Balance Score Matching

Although DBNF cannot be directly used for generative sampling, it can be combined with standard approaches such as score matching. Suppose that samples from a multi-modal target distribution $\mu(x)$ are available, and we seek a fast sampling method. As before, we choose μ_0 as a tractable adaptive prior and train a flow map F to minimize the DB loss $\text{DB}(\mu_0 \| F_{\#}\mu)$. Once μ_0 and F are trained, the distributions μ_0 and $F_{\#}\mu$ are close, and the remaining gap can be bridged by a standard generative layer such as score-based diffusion, which is able to overcome the geometric constraints of diffeomorphisms.

The central challenge remains the evaluation of the DB loss (11). In this setting, samples from $\mu(x)$ are available, so importance sampling is not needed. The main difficulty is that the gradient $\nabla U(x)$ is not directly accessible from samples alone.

Nevertheless, we can employ implicit score matching to obtain an approximation to $\nabla U(x)$. Let $s(x) \in \mathbb{R}^d$ denote a parametric score model. The implicit score matching loss is

$$\int_{\mathbb{R}^d} \mu(x) |s(x) - \nabla U(x)|^2 dx = \int_{\mathbb{R}^d} \mu(x) (|s(x)|^2 + 2\nabla \cdot s(x)) dx, \quad (21)$$

where the right-hand side depends only on samples from $\mu(x)$ and does not require explicit knowledge of $U(x)$. Minimizing (21) yields $s(x) \approx \nabla U(x)$, which can then be substituted into the DB loss (11). With this surrogate gradient, we train the flow map F and the adaptive prior $\mu_0(x)$ as in DBNF. Finally, a score-based diffusion layer is added between μ_0 and $F_{\#}\mu$ to close the remaining gap. Since μ_0 and $F_{\#}\mu$ are already close after DBNF training, the diffusion process only needs to run for a short time, which is expected to yield efficient and high-quality generative sampling.

References

Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020.

- Durk P Kingma and Prafulla Dhariwal. Glow: Generative flow with invertible 1x1 convolutions. *Advances in neural information processing systems*, 31, 2018.
- Polina Kirichenko, Pavel Izmailov, and Andrew G Wilson. Why normalizing flows fail to detect out-of-distribution data. *Advances in neural information processing systems*, 33:20578–20589, 2020.
- Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *International Conference on Learning Representations*, 2023.
- Frank Noé, Simon Olsson, Jonas Köhler, and Hao Wu. Boltzmann generators: Sampling equilibrium states of many-body systems with deep learning. *Science*, 365(6457):eaaw1147, 2019.
- George Papamakarios, Theo Pavlakou, and Iain Murray. Masked autoregressive flow for density estimation. *Advances in neural information processing systems*, 30, 2017.
- George Papamakarios, David Sterratt, and Iain Murray. Sequential neural likelihood: Fast likelihood-free inference with autoregressive flows. In *The 22nd international conference on artificial intelligence and statistics*, pages 837–848. PMLR, 2019.
- Danilo Rezende and Shakir Mohamed. Variational inference with normalizing flows. In *International conference on machine learning*, pages 1530–1538. PMLR, 2015.
- Marco Rudolph, Bastian Wandt, and Bodo Rosenhahn. Same same but DifferNet: Semi-supervised defect detection with normalizing flows. In *Proceedings of the IEEE/CVF winter conference on applications of computer vision*, pages 1907–1916, 2021.
- Yang Song, Jascha Sohl-Dickstein, Diederik P Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. In *International Conference on Learning Representations*, 2021.
- Hao Wu, Jonas Köhler, and Frank Noé. Stochastic normalizing flows. *Advances in neural information processing systems*, 33:5933–5944, 2020.